# Object-Oriented Programming and the Binomial Model

FRE6883 – Recitation 4

Keep state valid, expose a small interface, reuse the CRR engine

# From separate variables to one financial object

State: $S_0, U, D, R$

$+$ operations that use that state

$\downarrow$

`BinomialTreeModel`

**By the end**

You can read and complete a C++ class, explain object construction and lifetime, and use a `BinomialTreeModel` inside the generic CRR pricer.

Each activity follows the same rhythm: read the code, predict its behavior, then explain the design choice.

# Why group model data and model operations?

```
double S0 = 106.0;
double U  = 1.15125;
double D  = 0.86862;
double R  = 1.00545;

double q = RiskNeutProb(U, D, R);
double S = S0 * std::pow(U, i)
              * std::pow(D, n - i);
```

The four values travel together through many functions. A wrong order or an invalid update can silently corrupt every result.

**Object-oriented step**

Store the state once and ask the object to perform operations that depend on it.

# 01 | Identify the class interface and implementation – Problem

```
class Rectangle {
private:
    double width;
    double height;

public:
    Rectangle(double w, double h);
    double area() const;
    void scale(double factor);
};
```

**Questions**

Which lines describe the object's state? Which lines form the public interface? Which function may change the state?

# 01 | Identify the class interface and implementation – Solution

```
// State: hidden from client code
double width;
double height;

// Public interface
Rectangle(double w, double h);
double area() const;       // reads state
void scale(double factor); // may modify state
```

**Class**

A user-defined type that combines data and operations.

**Object**

One instance of that type, with its own state.

**Interface**

The public operations client code is allowed to call.

**GUIDED CODE** Encapsulation — Mark each line valid or invalid

```
Rectangle box(4.0, 3.0);

std::cout << box.area() << std::endl;  // A
box.scale(2.0);                        // B
box.width = 10.0;                      // C
std::cout << box.height << std::endl;  // D
```

**Question**

Which statements compile? Why might the two rejected statements be useful restrictions rather than inconveniences?

# 02 | Predict the access-control errors – Solution

```
std::cout << box.area(); // valid: public
box.scale(2.0);          // valid: public

box.width = 10.0;        // error: private
std::cout << box.height; // error: private
```

**Encapsulation**

Client code requests an operation through the public interface. The class controls how its private state is read or changed.

This gives the class one place to enforce rules such as positive dimensions or valid market inputs.

# 03 | Complete the constructor – Problem

**GUIDED CODE** Initialization list

Complete before revealing

```
class Rectangle {
private:
    double width;
    double height;
public:
    Rectangle(double w, double h)
        : /* initialize width and height */
    {
    }
};
```

**Questions**

When is a constructor called? Does it have a return type? What should `Rectangle box(4.0, 3.0);` store?

# 03 | Complete the constructor – Solution

```
Rectangle(double w, double h)
    : width(w), height(h)
{
}

Rectangle box(4.0, 3.0);
```

**Name**

A constructor has the same name as the class.

**Return type**

There is none, not even `void`.

**Initialization list**

Members are initialized before the constructor body begins.

# 04 | Which constructor runs? – Problem

**GUIDED CODE** Object initialization

Match each line to a constructor

```
class Rectangle {
public:
    Rectangle();
    Rectangle(double w, double h);
    Rectangle(const Rectangle& other);
};

Rectangle a;              // A
Rectangle b(4.0, 3.0);    // B
Rectangle c = b;          // C
Rectangle d(b);           // D
```

**Question**

Which line calls the default, parameterized, or copy constructor?

# 04 | Which constructor runs? – Solution

| Statement | Constructor | Meaning |
|---|---|---|
| `Rectangle a;` | default | no arguments |
| `Rectangle b(4.0, 3.0);` | parameterized | use supplied state |
| `Rectangle c = b;` | copy | create from `b` |
| `Rectangle d(b);` | copy | create from `b` |

**Do not confuse construction and assignment**

`Rectangle c = b;` creates `c`. A later `c = b;` assigns to an object that already exists.

# 05 | Find the missing `const` – Problem

**GUIDED CODE** Const member functions

Predict the compiler's complaint

```
class Rectangle {
public:
    double area() { return width * height; }
private:
    double width;
    double height;
};

void printArea(const Rectangle& box)
{
    std::cout << box.area() << std::endl;
}
```

**Question**

Why can the call to `area()` fail even though `area()` does not visibly modify the object?

# 05 | Find the missing `const` – Solution

```
class Rectangle {
public:
    double area() const
    {
        return width * height;
    }
};
```

The trailing `const` promises that the function will not change the object's observable state.

**Practical rule**

A `const` object or `const` reference may call only member functions declared `const`.

# 06 | Choose . or -> – Problem

**GUIDED CODE** Selecting members — Fill the two blanks

```
Rectangle box(4.0, 3.0);
Rectangle* ptr = &box;

std::cout << box /* ? */ area() << std::endl;
std::cout << ptr /* ? */ area() << std::endl;
```

**Questions**

Which operator selects a member through an object? Which selects it through a pointer to an object?

# 06 | Choose . or -> – Solution

```
std::cout << box.area() << std::endl;
std::cout << ptr->area() << std::endl;

// Equivalent, but harder to read:
std::cout << (*ptr).area() << std::endl;
```

**Read the type first**

Use . with an object or reference. Use -> with a pointer. The arrow combines dereference and member selection.

# 07 | Predict object lifetime – Problem

**GUIDED CODE** Scope and destructor

Write the event order

```
class Trace {
public:
    Trace(int id_) : id(id_) { std::cout << "+" << id; }
    ~Trace() { std::cout << "-" << id; }
private:
    int id;
};

int main()
{
    Trace a(1);
    { Trace b(2); }
    Trace c(3);
}
```

# 07 | Predict object lifetime – Solution

```
// Output, with no spaces:
+1+2-2+3-3-1
```

**Construction**

Each local object is constructed when execution reaches its declaration.

**Destruction**

Local objects are destroyed when their scope ends, in reverse construction order.

# 08 | Find the lifetime bug – Problem

**GUIDED CODE** Dynamic objects — Find the missing operation

```
Rectangle* box = new Rectangle(4.0, 3.0);
std::cout << box->area() << std::endl;

// program continues ...
```

**Questions**

Where does the object live? When is its destructor called? What must be added when the object is no longer needed?

# 08 | Find the lifetime bug – Solution

```
Rectangle* box = new Rectangle(4.0, 3.0);
std::cout << box->area() << std::endl;

delete box;
box = nullptr;
```

**Ownership rule for this course example**

Every successful `new` must eventually have one matching `delete`. Deleting the object calls its destructor and releases its memory.

Prefer ordinary local objects when dynamic lifetime is unnecessary.

# 09 | Complete the shared object counter – Problem

**GUIDED CODE** Static data member — One value shared by all objects

```
class Rectangle {
public:
    Rectangle() { /* increment count */ }
    ~Rectangle()  { /* decrement count */ }
    static int getCount() { return count; }
private:
    static int count;
};

/* define count outside the class */
```

**Question**

Why should `count` be `static` rather than stored separately in each object?

# 09 | Complete the shared object counter – Solution

```
class Rectangle {
public:
    Rectangle() { ++count; }
    ~Rectangle() { --count; }
    static int getCount() { return count; }
private:
    static int count;
};

int Rectangle::count = 0;
```

There is one `count` for the class, not one per object. Call it without an object:

`Rectangle::getCount()`

# Checkpoint: read a class in four passes

1. **State:** What data does each object store? What data is shared?
2. **Interface:** What may client code call? What stays private?
3. **Constness:** Which operations only observe state?
4. **Lifetime:** Which constructor creates the object? When is it destroyed?

**Apply the checklist now**

We will use it on the class from Topic 4: `fre::BinomialTreeModel`.

# 10 | Design the binomial model class – Problem

**GUIDED CODE** State and interface — Classify each item

```
namespace fre {
class BinomialTreeModel {
private:
    /* four market inputs */
public:
    /* q = (R - D) / (U - D) */
    /* S(n,i) = S0 * U^i * D^(n-i) */
    /* validate inputs */
    /* read S0, U, D, R */
};
}
```

**Task**

Name the private data members and propose the public member functions needed by the CRR pricer.

# 10 | Design the binomial model class – Solution

```
namespace fre {
class BinomialTreeModel {
private:
    double S0, U, D, R;
public:
    double RiskNeutProb() const;
    double CalculateAssetPrice(int n, int i) const;
    int ValidateInputData() const;

    double GetS0() const { return S0; }
    double GetU()  const { return U;  }
    double GetD()  const { return D;  }
    double GetR()  const { return R;  }
};
}
```

# 11 | Complete the constructors – Problem

**GUIDED CODE** Initialization

Use the member initialization list

```
BinomialTreeModel()
    : /* initialize all four values to zero */
{
}

BinomialTreeModel(double S0_, double U_,
                  double D_, double R_)
    : /* initialize from the arguments */
{
}
```

**Question**

Why do the argument names end in an underscore in the lecture code?

# 11 | Complete the constructors – Solution

```
BinomialTreeModel()
    : S0(0), U(0), D(0), R(0)
{
}

BinomialTreeModel(double S0_, double U_,
                  double D_, double R_)
    : S0(S0_), U(U_), D(D_), R(R_)
{
}
```

**Naming convention**

S0_ names the parameter; S0 names the data member. The underscore makes the mapping explicit.

# 12 | Complete the out-of-class definitions – Problem

**GUIDED CODE** Scope resolution

Add the missing class scope

```
double /* ? */ RiskNeutProb() const
{
    return (R - D) / (U - D);
}

double /* ? */ CalculateAssetPrice(int n, int i) const
{
    return S0 * std::pow(U, i)
              * std::pow(D, n - i);
}
```

# 12 | Complete the out-of-class definitions – Solution

```
double BinomialTreeModel::RiskNeutProb() const
{
    return (R - D) / (U - D);
}

double BinomialTreeModel::CalculateAssetPrice(int n, int i) const
{
    return S0 * std::pow(U, i)
              * std::pow(D, n - i);
}
```

**The :: operator**

It says these function definitions belong to `BinomialTreeModel`; they may therefore access `S0`, `U`, `D`, and `R`.

# 13 | Find the validation bug – Problem

**GUIDED CODE** No-arbitrage input check Test boundary cases

```
int BinomialTreeModel::ValidateInputData() const
{
    if (S0 <= 0.0 || U <= 0.0 ||
        D <= 0.0 || R <= 0.0)
        return 1;

    if (R >= U || U <= D)
        return 1;

    return 0;
}
```

**Question**

The model requires $0 < D < R < U$. Which invalid case is not detected by the second condition?

# 13 | Find the validation bug – Solution

```
int BinomialTreeModel::ValidateInputData() const
{
    if (S0 <= 0.0 || U <= 0.0 ||
        D <= 0.0 || R <= 0.0)
        return 1;

    if (R >= U || R <= D)
        return 1;

    return 0;
}
```

**Invariant**

The class accepts only positive inputs satisfying $D < R < U$. For those inputs, $q = (R - D)/(U - D)$ lies strictly between 0 and 1.

# 14 | Create and query a model – Problem

**GUIDED CODE** Object use — Complete the member calls

```
fre::BinomialTreeModel model(
    106.0, 1.15125, 0.86862, 1.00545);

double q = /* ask model for q */;
double S32 = /* ask model for S(3,2) */;

std::cout << q << " " << S32 << std::endl;
```

**Predict**

Should client code need to pass `U`, `D`, or `R` again when it asks for `q`?

# 14 | Create and query a model – Solution

```
fre::BinomialTreeModel model(
    106.0, 1.15125, 0.86862, 1.00545);

double q = model.RiskNeutProb();
double S32 = model.CalculateAssetPrice(3, 2);

std::cout << q << " " << S32 << std::endl;
```

**The object supplies its own state**

The caller specifies only the node coordinates $(n, i)$. The object combines them with the stored market inputs.

# 15 | Why pass the model by `const` reference? – Problem

**GUIDED CODE** Function interface — Choose among value, reference, const reference

```
double* PriceByCRR(
    const BinomialTreeModel& Model,
    int N,
    double K,
    double (*Payoff)(double z, double K));
```

**Questions**

Why `&`? Why `const`? Which member functions must be `const` for this parameter to work?

# 15 | Why pass the model by `const` reference? – Solution

**Reference**

The function uses the original model object without making a copy.

**Const**

The function promises not to change that object.

**Required member interface**

`RiskNeutProb()`, `CalculateAssetPrice()`, and `GetR()` must all be declared `const`.

Read the type aloud: "a read-only reference to a `BinomialTreeModel`."

# 16 | Build terminal payoffs with the object – Problem

**GUIDED CODE** CRR initialization

Complete the two member calls

```
double* Price = new double[N + 1];

for (int i = 0; i <= N; ++i)
{
    double z = /* asset price at node (N,i) */;
    Price[i] = Payoff(z, K);
}

double q = /* risk-neutral probability */;
```

**Question**

What knowledge stays inside the model, and what behavior remains supplied by the payoff function pointer?

# 16 | Build terminal payoffs with the object – Solution

```
double* Price = new double[N + 1];

for (int i = 0; i <= N; ++i)
{
    double z = Model.CalculateAssetPrice(N, i);
    Price[i] = Payoff(z, K);
}

double q = Model.RiskNeutProb();
```

**Division of responsibility**

The model knows the tree dynamics. The payoff function knows the contract. `PriceByCRR` knows the backward-induction algorithm.

# 17 | Complete backward induction – Problem

**GUIDED CODE** Object inside the CRR loop — Fill the discount factor

```
for (int n = N - 1; n >= 0; --n)
{
    for (int i = 0; i <= n; ++i)
    {
        Price[i] =
            (q * Price[i + 1]
             + (1.0 - q) * Price[i])
            / /* read R from Model */;
    }
}
```

**Explain**

Why can `Price[i]` be overwritten in place? Why is the loop order unchanged from Topic 2?

# 17 | Complete backward induction – Solution

```
for (int n = N - 1; n >= 0; --n)
{
    for (int i = 0; i <= n; ++i)
    {
        Price[i] =
            (q * Price[i + 1]
             + (1.0 - q) * Price[i])
            / Model.GetR();
    }
}
```

The class changes how the market inputs are packaged. It does not change the CRR recurrence or its $O(N^2)$ running time.

# 18 | Complete the pricing driver – Problem

**GUIDED CODE** Integration — Wire the object to the generic engine

```
int main()
{
    fre::BinomialTreeModel Model(
        106.0, 1.15125, 0.86862, 1.00545);
    int N = 8;
    double K = 100.0;

    double* call = /* price the call */;
    double* put  = /* price the put */;

    std::cout << call[0] << " " << put[0];
    delete[] call;
    delete[] put;
}
```

Call the same pricing function twice, changing only the payoff function.

# 18 | Complete the pricing driver – Solution

```cpp
int main()
{
    fre::BinomialTreeModel Model(
        106.0, 1.15125, 0.86862, 1.00545);
    int N = 8;
    double K = 100.0;

    double* call = fre::PriceByCRR(
        Model, N, K, fre::CallPayoff);
    double* put = fre::PriceByCRR(
        Model, N, K, fre::PutPayoff);

    std::cout << call[0] << " " << put[0] << std::endl;
    delete[] call;
    delete[] put;
}
```

**Expected output:** call $\approx 21.68$ and put $\approx 11.43$.

# 19 | Diagnose four class mistakes – Problem

**GUIDED CODE** Fast final check — One minute, four fixes

```
const BinomialTreeModel Model(100.0, 1.1, 0.9, 1.02);

double q = Model.RiskNeutProb(); // RiskNeutProb lacks const
Model.S0 = 105.0;                // S0 is private

BinomialTreeModel* p = &Model;   // pointer is non-const
std::cout << p.GetR();           // wrong selector
```

**Task**

Identify the issue on each annotated line and state the corrected declaration or expression.

# 19 | Diagnose four class mistakes – Solution

```cpp
// Member function promises not to modify the object:
double RiskNeutProb() const;

// Update only through an allowed public operation:
// Model.UpdateBinomialTreeModel(...); // impossible here: Model is const

// A pointer to a const object, selected with ->
const BinomialTreeModel* p = &Model;
std::cout << p->GetR();
```

**Exit ticket**

For one member function, say: what state it uses, whether it modifies that state, and whether client code should be allowed to call it.

# Reserve exercises

# R1 | Complete the `Time` class – Challenge

**GUIDED CODE** Previous-year workshop — Static count and object lifetime

```
class Time {
public:
    Time(int h, int m, int s);
    Time(const Time& other);
    ~Time();
    void addSeconds(int seconds);
    void print() const;
    static int getObjectCount();
private:
    int hours, minutes, seconds;
    static int objectCount;
};
```

Which three functions change `objectCount`? How must `addSeconds` handle crossings of 60 and 3600 seconds?

# R1 | Complete the `Time` class – Solution

```cpp
int Time::objectCount = 0;

Time::Time(int h, int m, int s)
    : hours(h), minutes(m), seconds(s) { ++objectCount; }

Time::Time(const Time& other)
    : hours(other.hours), minutes(other.minutes),
      seconds(other.seconds) { ++objectCount; }

Time::~Time() { --objectCount; }

void Time::addSeconds(int value)
{
    int total = 3600 * hours + 60 * minutes + seconds + value;
    hours = (total / 3600) % 24;
    minutes = (total / 60) % 60;
    seconds = total % 60;
}
```

# R2 | Why is the default copy unsafe? – Challenge

**GUIDED CODE** Previous-year workshop Pointer ownership

```
class Account {
public:
    Account(const char* name, double balance);
    ~Account();
private:
    char* owner;
    double balance;
};

Account first("Ada", 1000.0);
Account second = first;
```

If only the pointer value is copied, what do both `owner` members point to? What happens when both destructors run?

# R2 | Why is the default copy unsafe? – Solution

```
Account::Account(const Account& other)
    : balance(other.balance)
{
    owner = new char[std::strlen(other.owner) + 1];
    std::strcpy(owner, other.owner);
}

Account::~Account()
{
    delete[] owner;
}
```

**Deep copy**

Each `Account` owns a separate character array. Copying duplicates the characters rather than sharing the same address.

This raw-memory pattern is included because it appears in the previous workshop; modern code would usually store the name in `std::string`.

# R3 | Add a digital call – Challenge

**GUIDED CODE** Previous-year workshop

Reuse the same model and CRR engine

```
double DigitalCallPayoff(double z, double K)
{
    /* return 1.0 if z > K, otherwise 0.0 */
}

fre::BinomialTreeModel Model(100.0, 1.1, 0.9, 1.02);
double* price = fre::PriceByCRR(
    Model, 3, 100.0, DigitalCallPayoff);
```

**Task**

Complete the payoff. Which part of the object-oriented pricer must change?

# R3 | Add a digital call – Solution

```cpp
double DigitalCallPayoff(double z, double K)
{
    return (z > K) ? 1.0 : 0.0;
}

double* price = fre::PriceByCRR(
    Model, 3, 100.0, DigitalCallPayoff);

std::cout << price[0] << std::endl; // about 0.610625
delete[] price;
```

**Reuse**

The model class and backward-induction engine stay unchanged. Only the payoff behavior is replaced.

# Recitation 4 cheat sheet

| Concept | Working rule |
|---|---|
| `private` | Hide state; preserve valid relationships between members. |
| `public` | Expose the small set of operations clients need. |
| Constructor | Initialize every data member, preferably in the initialization list. |
| Trailing `const` | Promise that a member function will not change the object. |
| `.` / `->` | Use `.` with an object; `->` with a pointer. |
| `Class::member` | Name a member outside the class or access a static member. |
| `const T&` | Read an existing object without copying or modifying it. |
| Destructor | Runs when an object dies; releases resources the object owns. |

# R4 | Represent an option contract

```
enum class OptionType { Call, Put };

class EuropeanOption {
public:
    EuropeanOption(double strike, double maturity,
                   OptionType type);
    double payoff(double spotAtMaturity) const;
    double strike() const;
    double maturity() const;
private:
    double K, T;
    OptionType type;
};
```

The contract owns its terms and payoff rule. A pricing model can read those terms without exposing or changing them.

**Design boundary**

Keep contract data in `EuropeanOption`; keep market assumptions and valuation methods in a model or pricer.

# R5 | Encapsulate Black–Scholes assumptions

```
class BlackScholesModel {
public:
    BlackScholesModel(double spot, double rate,
                      double dividendYield,
                      double volatility);
    double callPrice(double K, double T) const;
    double putPrice(double K, double T) const;
private:
    double S0, r, q, sigma;
};
```

The object groups current spot, continuously compounded rate, dividend yield, and volatility. Its pricing methods use the closed-form European-option formulas.

$$d_1 = \frac{\log(S_0/K) + (r - q + \sigma^2/2)T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T}.$$

The formula relies on constant volatility and rates, and lognormal risk-neutral stock dynamics.

# R6 | Make a local volatility surface a class

```
class LocalVolatilitySurface {
public:
    LocalVolatilitySurface(
        std::vector<double> spotGrid,
        std::vector<double> timeGrid,
        std::vector<std::vector<double>> values);

    double volatility(double spot, double time) const;
private:
    std::vector<double> spots, times;
    std::vector<std::vector<double>> sigma;
};
```

The public query hides the grid layout and interpolation rule. A Monte Carlo pricer or finite-difference solver can ask for $\sigma(S, t)$ through the same interface.

**Design questions**

Which grid and value dimensions must match? How should the class handle out-of-grid queries? What checks should reject invalid volatilities?